

AIDL — AI Directive Language

A Specification for Machine-Readable AI Agent Governance

Niels Goldstein

Defensive Publication · June 1, 2026 · MIT License · github.com/nielsg2/aidl-spec

Abstract

AIDL (AI Directive Language) is a compact, machine-readable language for expressing the rules that govern the behavior of AI agents, paired with a multi-agent governance model. It lets the policies that constrain an AI system — roles, permissions, escalation paths, reputation thresholds, and audit requirements — be declared in a form that is simultaneously human-readable and machine-parseable, so governance travels with the system rather than residing in documentation or operator memory. This disclosure describes the AIDL file format, a set of operational directives expressed in it, and a governance model in which heterogeneous AI agents are assigned mutable roles, accumulate reputation from observed behavior, and resolve conflicts through reputation-weighted adjudication with cross-provider independence weighting. It is published openly as a defensive publication and a contribution to the public record of prior art.

Background and Problem

Autonomous and semi-autonomous AI agents are increasingly delegated authority to take consequential actions. The governance of such agents — defining what each agent may do, how disagreements between agents are resolved, and how a degraded or misbehaving agent is detected and constrained — is frequently implicit, ad hoc, or buried in application code. As a result, an AI system's trust posture is neither auditable nor portable. AIDL treats governance as declarative, machine-parseable data: a Zero Trust posture for AI agents, in which an agent's authority is earned through observed behavior and is continuously revocable, rather than granted statically and assumed to persist.

The AIDL Language

An AIDL file (.aidl) is the compact, operational companion to a longer human-language directive document. It is designed to be human-readable, machine-parseable, and small enough to inject into a session as persistent context. It has two layers.

Layer A — Operational configuration

Key/value blocks (YAML-style) that set the governance posture; an enforcing runtime reads them as policy.

```
OUTPUT:
  HEADERS: REQUIRED
  AUTO_REGEN: [README.md, CHANGELOG]
GIT:
  ALLOW_COMMIT: ONLY_ON "<explicit trigger phrase>"
  VERSION_TAG_REQUIRED: TRUE
EXECUTION:
  JSON_TASKS: ATOMIC
  CONTEXT_REQUIRED: TRUE
LOGGING:
  FORMAT: STRUCTURED
  DB_TARGET: SQLITE_READY
```

CONTRIBUTORS:
TRACK_AUTHORS: TRUE
SCORE_REPUTATION: TRUE

Layer B — Directive index

A numbered list of directives, each one line, encoded as D###. [Tag] description, where D### is a stable identifier (D000 reserved for a meta-directive) and [Tag] classifies the directive. Each line is the enforceable summary of a fuller directive in the canonical source.

```
D000. [Meta]      Maximum-fidelity execution; regenerate from source; no partial output.
D007. [Memory]    Reload directive and roadmap files at session start; assume context degraded.
D033. [Behavior]  Multi-agent role enforcement: Orchestrator / Executor / Arbiter.
D035. [Memory]    Agent reputation and logging; reputation drives role mutation.
D036. [Security]  Deterministic write-scoped execution; read-only default; pre-write structural map
```

Tags: [Meta] governs the directive system; [Behavior] how the agent acts; [Memory] context and continuity; [Persistence] files and repositories; [Interaction] human/agent protocol; [Output] artifact format and integrity; [Security] scope, drift, audit; [Extensions] registries and reusable capability.

Design principles: the rules are data, not prose; identifiers are stable across revisions so logs and references remain valid; the compact form points at a canonical form; least privilege (read-only) by default.

Multi-Agent Governance Model

The governance model lets a set of AI agents — potentially running on heterogeneous providers — be collectively more trustworthy than any single agent. Authority is earned, measured, and revocable.

Roles

Agents are assigned roles, which are assignments rather than identities (see Role Mutation):

- **Orchestrator** — coordinates workflow, maintains global context, sequences tasks, enforces constraints, logs interactions, routes disputes.
- **Executor** — performs the work within an explicit write scope; emits structured, auditable output.
- **Arbiter** — adversarially reviews Executor output; flags contradictions, drift, and ungrounded assumptions; critiques only, does not modify artifacts.

Reputation

Each agent accumulates a reputation score from observed task outcomes, with components including accuracy, determinism, constraint adherence, error rate, drift detection, and latency. Reputation observations decay over time, so recent behavior weighs more heavily than accumulated history.

Cross-provider independence weighting

When agents run on different underlying models, their judgments are not fully independent: two agents on the same base model share architecture, training data, and failure modes, so their agreement is partly correlated rather than corroborating. When combining agent judgments, correlation between agents that share a provider, model family, or training lineage is discounted. Two genuinely independent agents corroborate more than two siblings of the same model.

Role mutation, adjudication, escalation

- **Role mutation:** when reputation crosses configured thresholds (sustained over a sliding window to suppress transient spikes), an agent's role changes — a reliable Executor may be promoted to Arbiter; a noisy Arbiter demoted.

- **Adjudication:** conflicting outputs are resolved by reputation-weighted voting, each vote weighted by accumulated reputation and the cross-provider independence factor.
- **Escalation:** ambiguous Executor output escalates to the Arbiter; flagged contradictions return a corrective directive from the Orchestrator; systemic drift triggers a full review; a failed determinism check applies a reputation penalty.

Safety primitives

- **Write-scoped execution:** read-only by default; writes require an explicit declared scope and a pre-write structural map; no hallucinated content to disk; all writes auditable and reversible.
- **Drift detection and halt:** on detected drift (content inconsistency, duplicate insertion, version regression, structural corruption, semantic conflict), the agent halts, emits a severity-rated drift report, and presents options without auto-resolving.
- **Mandatory logging:** every interaction, role change, adjudication, and escalation is logged. Reputation is computed from this log, making the log a governed, integrity-sensitive (optionally signed) artifact.

Directive Set (Summary)

The disclosed directive set spans D000–D047, grouped as follows. The complete text is in the published repository.

- **Meta (D000):** maximum-fidelity execution, with an explicit clause that directives cannot override a model's safety guardrails and that an agent must state its real limits.
- **Core (D001–D025):** executable-in-reality output, complete-file delivery, structured logging and headers, archival-before-overwrite, session reload and rehydration, output integrity and completion locking.
- **Extended (D026–D032):** execution-reality compliance, human-as-orchestrator, command-first delivery, user sovereignty, commit-time mandates, contributor tracking.
- **Multi-Agent Governance (D033–D037):** role enforcement, adjudication and escalation, reputation and logging, write-scoped execution, drift detection and halt.
- **Operational Frameworks (D038–D047):** deterministic execution, workflow continuity, file-system-aware operations, schema extraction, canonicalization, DAG construction, meta-orchestrator contracts, patch generation, large-document consolidation, workflow integration.

Provenance and Open-Publication Statement

The directive framework first appeared under version control on June 8, 2025, and the `.aidl` language was named on June 25, 2025, with development recorded in independent, timestamped source-control history. This work was not developed in isolation from the field, and claims no primacy: through 2025–2026 many independent efforts converged on machine-readable runtime governance for AI agents — academic domain-specific languages, industry risk taxonomies, and vendor toolkits among them; some predate this work and most followed it. It is released as a contribution and as prior art, under the MIT License, so the underlying ideas remain free for anyone to build on.

License

Copyright (c) 2025 Niels Goldstein. Released under the MIT License. Permission is granted, free of charge, to any person obtaining a copy of this work, to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies, subject to inclusion of the copyright and permission notice. The work is

provided "as is", without warranty of any kind.